

CSC 487 HW 1 Report – Tree Classifier

Code Explanation

Before training the models on the tree data, I projected its 426 hyperspectral features into only 32 using PCA as instructed in order to make the learning process easier. I then used sklearn to train a linear, multi-class decision model (using LogisticRegression), along with a neural network using a single hidden layer with 100 neurons (using sklearn.neural_network.MLPClassifier). I then used sklearn's built-in fit and score functions to train and test the model respectively.

When increasing the max number of iterations during the NN training, this only decreased the NN accuracy on the test set. This suggested that during the training process the NN was overfitting to the training data. Thus, I kept the number of max training iterations high but significantly increased the alpha parameter in order to increase the magnitude of regularization in an attempt to reduce the higher order terms' coefficients and thus reduce overfitting, which seemed to work, boosting the accuracy above the linear model's, being about 85% and 83% respectively. The learning rate of the NN was also settled at 0.0025 along with 1000 max epochs, as this was found through trial and error to lead to higher test accuracies in this case.

For the Pytorch linear and NN implementations, I wrote a standard accuracy scoring function which simply counted the number of correct predictions made by the model and divided that by the total number of data points. I then wrote a training function that used batches utilizing pytorch's DataLoader class and the standard training loop of zeroing the gradients, calculating model outputs, running the loss backward, and running the optimizer step to apply the (stochastic) gradient descent to update the weights. For the Pytorch NN, the SGD optimizer, learning rate, CrossEntropyLoss function, and weight decay were all used according to the assignment.

I mostly used the sklearn and pytorch documentation/APIs for guidance when completing this assignment, especially the examples for how the different classes and functions I needed were used, as well as which classes and functions were even available. For example, using the sklearn docs to understand the difference between PCA.fit() and PCA.transform, or looking at the examples in the torch.nn.Sequential() docs when implementing my models with PyTorch. That being said, I took a glance at Professor Ventura's paper on this topic and the linked Github with the code used in order to double check that I was applying my PCA correctly, as the dimensions of the matrix returned by the

PCA.fit() function initially caused me some confusion. I also used Perplexity (AI) for double checking how to use torch.nn.Sequential() for just a linear multiclass classifier, and further explanation of several things like what loss_fn.backward() is doing under the hood, among other things.

Discussion

The raw data provided had the following shapes: Train Features was 15707x426 of int16, Train Labels was 15707x1 of type uint8, Test Features was 1554x426 of int16, and Test Labels was 1554x1. This means that the raw data had 426 features (which were the arrays' columns), and that there were 15707 training examples (91% of the total data) and 1554 test points (9% of the total data), corresponding to the arrays' number of rows. The labels only had 1 column per example, which was which tree species. The labels ranged from 0-7, meaning there were 8 different tree species options for the classifier. The features each ranged from 0 up to an integer in the thousands, corresponding the emitted strength in that spectrum band from the tree. The highest feature maximum went up to 14998, whereas the lowest feature maximum was only 388.

The four models' results were as follows:

- Sklearn Models
 - Linear
 - Training Accuracy: 0.85535
 - Test Accuracy: 0.83398
 - NN
 - Training Accuracy: 0.96931
 - Test Accuracy: 0.85135
- PyTorch Models
 - Linear
 - Training Accuracy: 0.84822
 - Test Accuracy: 0.82754
 - NN
 - Training Accuracy: 0.92118
 - Test Accuracy: 0.86229

From these accuracies, we can see that there appears to be some degree of overfitting on all models, as the training accuracy was higher than the test accuracy for all four. While minor on the linear models (each only having ~2% difference between training set and test

set), this was definitely significant for the NNs for both libraries even when utilizing regularization to try and manage the overfitting. While both NNs seemed to overfit their training data, they still both performed 2-3% better than the linear classifiers. The sklearn NN seemed to overfit more as it had a higher training accuracy and lower test accuracy than pytorch's NN. This was even the case despite having used an alpha of 0.1 compared to torch's weight_decay of only 0.001, indicating perhaps these hyperparameters are not used in the exact same way for regularization when training their respective NNs.